Openclaw Voice Assistant

Openclaw Voice Assistant
1. Function Overview
2. Hardware Connection
3. Environment Setup
4. Modify Program Configuration
5. Enabling HTTP Functionality in OpenClaw
6. Running the Program and Demonstration

1. Function Overview

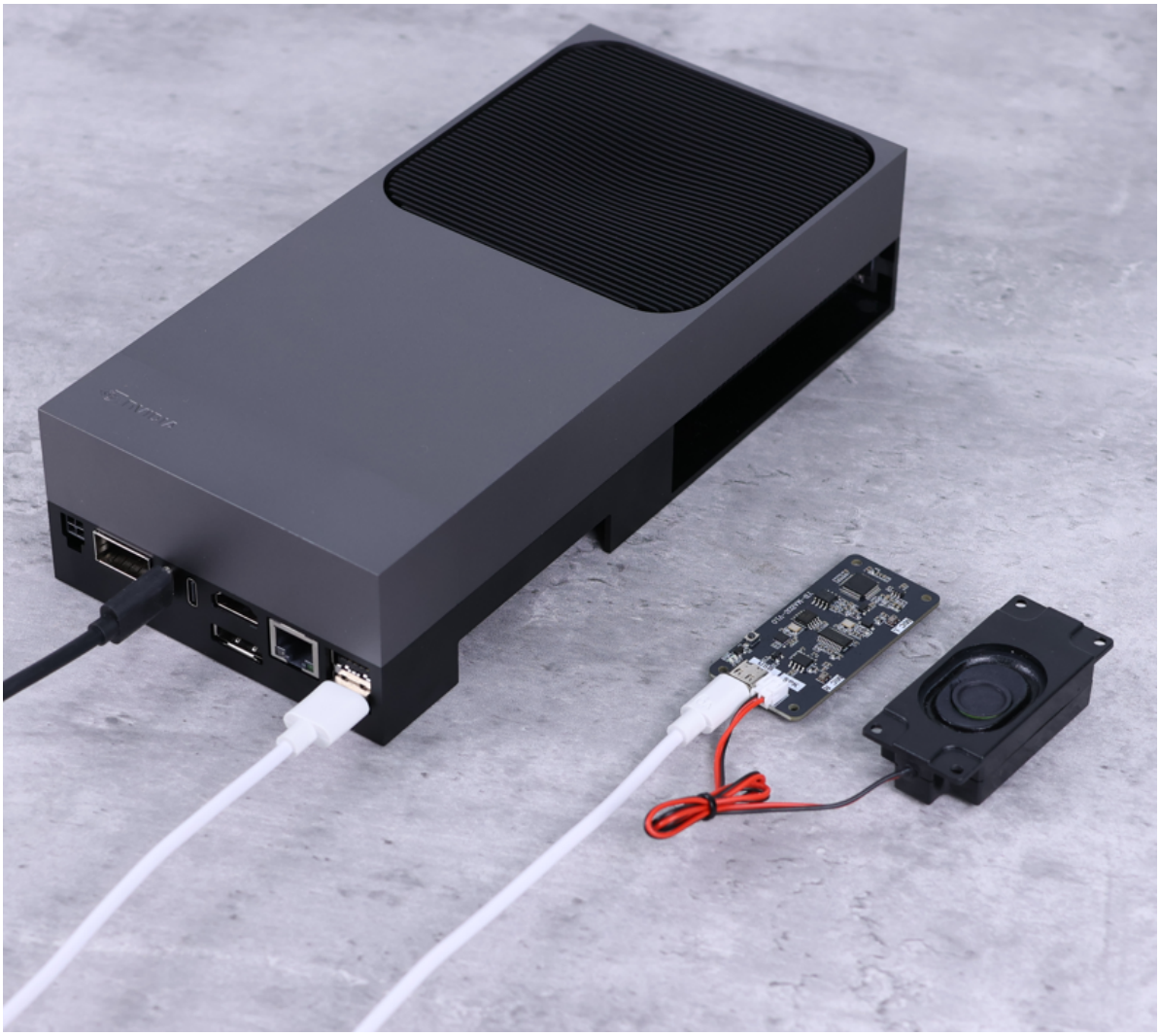
The Openclaw voice assistant uses a speech module to capture sound, converts the sound into text, sends it to Openclaw, receives Openclaw's responses, and finally converts the responses back into audio for playback.

The speech-to-text and text-to-speech functions use the qwen3-asr-flash and qwen3-tts-flash models from Tongyi Qianwen, so you need to fill in the Alibaba Cloud Bailian Platform API_KEY to use it.

Openclaw needs to be configured in advance, either online or offline.

2. Hardware Connection

This example requires connecting not only the power adapter but also the large language model module for language input and output, and a DP monitor, mouse, and keyboard for displaying the interface and performing operations, or logging into a remote desktop.



3. Environment Setup

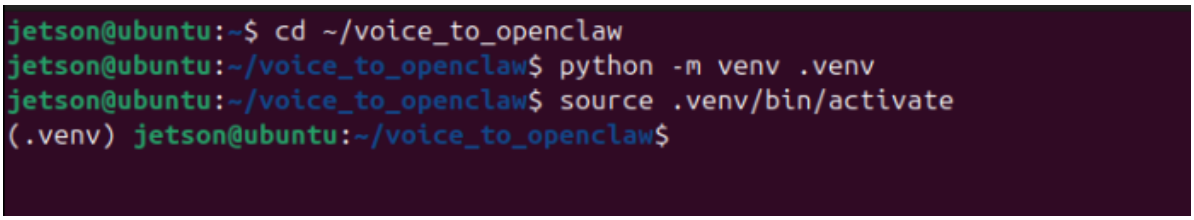
The factory image has been successfully set up.

Open the terminal, transfer the voice_to_openclaw.zip file from the program source code collection to the motherboard's user directory and extract it.

```
Unzip voice_to_openclaw.zip
```

Run the following command to create a new Python virtual environment

```
cd ~/voice_to_openclaw
python -m venv .venv
source .venv/bin/activate
```



Install the relevant Python dependencies

```
pip install -r requirements.txt
```

```
jetson@ubuntu:~/voice_to_openclaw$ source .venv/bin/activate
(.venv) jetson@ubuntu:~/voice_to_openclaw$ pip install -r requirements.txt
Collecting sounddevice>=0.4.7 (from -r requirements.txt (line 1))
  Using cached sounddevice-0.5.5-py3-none-any.whl.metadata (1.4 kB)
Collecting numpy>=1.26.0 (from -r requirements.txt (line 2))
  Using cached numpy-2.4.4-cp312-cp312-manylinux_2_27_aarch64.manylinux_2_28_
ch64.whl.metadata (6.6 kB)
```

4. Modify Program Configuration

Open the terminal, copy the example configuration as the valid configuration. The program will read the configuration parameters in the .env file when it runs.

```
cd ~/voice_to_openclaw
cp env.example .env
```

Open the `.env` file:

```
vim .env
```

You *must* fill in the value for `DASHSCOPE_API_KEY`; this value corresponds to your API Key for Tongyi Qianwen. For detailed instructions on how to obtain this key, please refer to the course section: [Online AI Large Model Development] -> [Online Large Models (Text Interaction)] -> [Tongyi Qianwen Large Model] to look up the information or register.

```
DASHSCOPE_API_KEY=
```

```
DASHSCOPE_API_KEY=sk-f8bf9c13***************cf8463d9
```

The following are optional configurations:

1. If you wish to switch to English conversation, change the value of the `DASHSCOPE_LANGUAGE` parameter to `en`.

```
DASHSCOPE_LANGUAGE=en
```

2. If you wish to change the Speech-to-Text (ASR) model, modify the value of the `DASHSCOPE_ASR_MODEL` parameter.

```
DASHSCOPE_ASR_MODEL=qwen3-asr-flash
```

3. If you wish to change the Text-to-Speech (TTS) model, modify the value of the `DASHSCOPE_TTS_MODEL` parameter.

```
DASHSCOPE_TTS_MODEL=qwen3-tts-flash
```

4. If you wish to change the TTS speaker (voice timbre), modify the value of the `DASHSCOPE_TTS_VOICE` parameter.

```
DASHSCOPE_TTS_VOICE=Cherry
```

The available voice options on the Tongyi Qianwen platform can be found and previewed at the following link:

```
https://help.aliyun.com/zh/model-studio/qwen-tts?mode=pure#bac280ddf5a1u
```

5. If you wish to change the acknowledgment phrase spoken after the voice wake-up command is detected, modify the value of the `WAKE_ACK_TEXT` parameter. Additionally, delete the `ack_tts.wav` file; a new `ack_tts.wav` file will be automatically generated the next time you run the program. If you do not require a spoken response after the wake-up command, simply leave the `WAKE_ACK_TEXT` parameter blank.

```
WAKE_ACK_TEXT=I'm here.
```

```
rm ~/voice_to_openclaw/ack_tts.wav
```

6. If you need to adjust the ambient noise filtering level, please modify the value of the `AGGRESSIVENESS` parameter. The valid range is [0, 1, 2, 3]; a higher value is suitable for environments with higher noise levels.

```
AGGRESSIVENESS=2
```

7. If you need to adjust the silence timeout duration that triggers the end of recording, please modify the value of the `MAX_SILENCE_MS` parameter. The unit is milliseconds; by default, 2 seconds of silence signals the end of the recording.

```
MAX_SILENCE_MS=2000
```

8. If you need to adjust the maximum recording duration (recording timeout), please modify the value of the `MAX_UTTERANCE_MS` parameter. The unit is milliseconds; by default, recording automatically ends 5 seconds after the wake-up command is detected.

```
MAX_UTTERANCE_MS=5000
```

9. If you need to adjust the timeout duration for OpenClaw, please modify the value of the `OPENCLAW_TIMEOUT` parameter. The unit is seconds; the default timeout is 3 minutes.

```
OPENCLAW_TIMEOUT=180
```

10. If you are not using the factory-provided system image, the system may not have the voice module port bound to `/dev/myserial`. In this case, please modify the `MIC_SERIAL_PORT` parameter to reflect the actual USB port name (e.g., `/dev/ttyUSB0`).

```
MIC_SERIAL_PORT=/dev/ttyUSB0
```

5. Enabling HTTP Functionality in OpenClaw

Since OpenClaw's HTTP functionality is disabled by default, you must add an HTTP enablement configuration to the OpenClaw settings to use it. If you have already added the HTTP enablement configuration, you may skip this step.

Open a terminal and run the following command to edit the `openclaw.json` file:

```
vim ~/.openclaw/openclaw.json
```

Search for `[gateway]`. If the `gateway` object does not contain an HTTP enablement configuration, add the necessary configuration settings within the `gateway` object. Then, save the configuration file.

```
    "http": {
      "endpoints": {
        "chatCompletions": {
          "enabled": true
        }
      }
    },
```

Restart the OpenClaw gateway to apply the configuration changes.

```
openclaw gateway restart
```

6. Running the Program and Demonstration

Open a terminal and activate the virtual environment:

```
cd ~/voice_to_openclaw
source .venv/bin/activate
```

Run the program:

```
python -m src.main
```

```
jetson@ubuntu:~$ cd ~/voice_to_openclaw
jetson@ubuntu:~/voice_to_openclaw$ source .venv/bin/activate
(.venv) jetson@ubuntu:~/voice_to_openclaw$ python -m src.main
[system] voice_to_openclaw started
[system] ASR_ENGINE=dashscope, asr_model=qwen3-asr-flash, asr_target=https://das
hscope.aliyuncs.com/api/v1, endpoint=http://127.0.0.1:18789/v1/chat/completions,
 publish_mode=gateway_chat, tts_enabled=True, tts_voice=Cherry
[wake] serial enabled, port=/dev/myserial, baudrate=115200
```

Say "Hi,yahboom" to wake up the device. Once awakened, the device will respond, "I'm here." At this point, the device begins recording. (The terminal will print "1" to indicate an active speaking state, or "-" to indicate a silent state. By default, recording automatically ends after 2 seconds of silence, or if the total recording duration exceeds the default maximum limit of 6 seconds.)

Then, state the command you want OpenClaw to execute—for example: "Tell me a joke."

Once the recording is complete, the ASR (Automatic Speech Recognition) module converts the audio into text and sends it to OpenClaw. After receiving a response from OpenClaw, the TTS (Text-to-Speech) module converts the content back into audio and plays it aloud.

Note: While audio content is being played back, you can interrupt the playback and issue a new command by saying "Hi,yahboom" to wake the device again.